Information Retrieval (COMPSCI5011) -- Comprehensive Exam Notes

University of Glasgow | School of Computing Science Exam: Friday 08/05/2026, 14:00 -- 1h 30m | CLOSED BOOK, No Calculator | Digital On Campus

Table of Contents

1. Introduction to IR

What is Information Retrieval?

Information Retrieval (IR) is the science of searching for information in documents, searching for documents themselves, and searching for metadata about documents. It deals with **unstructured data** (primarily text).

- **Salton (1968):** "IR is concerned with the structure, analysis, organization, storage, searching and retrieval of information."
- **Key distinction:** IR deals with *unstructured* data (text, images, video) vs. databases that handle *structured* data with exact matching.

Key Concepts

Concept	Definition
Information Need	The underlying reason a user submits a query
Query	The user's formulation of their information need
Relevance	Whether a document contains the information the user sought
Topical Relevance	Document and query are about the same topic
User Relevance	Takes into account usefulness, novelty, freshness, etc.

IR vs. Database Retrieval

Feature	IR	Database
Data	Unstructured (text)	Structured (tables)
Matching	Best-match (ranked)	Exact match
Results	Ranked by relevance	Exact set
Query	Natural language / keywords	SQL / formal

Modern IR: RAG (Retrieval-Augmented Generation)

IR plays a crucial role in modern AI through **RAG** = Large Language Models + IR. The IR component retrieves relevant documents that provide context for LLM generation.

> **Exam Tip:** Be able to define IR, distinguish it from database retrieval, and explain why ranked retrieval is preferred over set retrieval.

---

2. IR System Architecture

The Retrieval Process

```
Information Need -> Query Formulation -> Text Processing -> Matching with Index -> Ranked R
                                                                                    |
                                                                                Relevance Feedback
```

Three main parts: 1. **Text Processing** (indexing & query processing) 2. **Indexing** (building data structures) 3. **Retrieval** (matching & ranking)

Text Processing Pipeline

```
Raw Text -> Tokenization -> Stop Word Removal -> Stemming/Lemmatization -> Index Terms
```

1. **Tokenization:** Breaking text into words/tokens
2. Handle punctuation, hyphens, case folding, numbers, special characters
3. Languages without spaces (Chinese, Japanese) need segmentation
4. **Stop Word Removal:** Remove high-frequency, low-information words
5. Examples: "the", "of", "and", "to", "is"
6. Reduces index size significantly
7. Trade-off: may hurt phrase queries ("to be or not to be")
8. **Stemming:** Reduce words to root form
9. **Porter Stemmer** (most common): rule-based suffix stripping
    - "computing" -> "comput", "computers" -> "comput"
10. **Lemmatization:** dictionary-based, returns actual words
    - "better" -> "good", "ran" -> "run"
11. Stemming can cause errors:
    - **Understemming:** failing to conflate related words
    - **Overstemming:** conflating unrelated words ("universe" and "university")

Bag of Words (BoW) Representation

- Treat document as an unordered collection of words
- Assign a weight to each term (e.g., frequency or binary presence)
- **Disregards word order, structure, and meaning**
- Simple yet effective!

The Inverted Index

The **inverted index** is the core data structure for IR systems.

Structure:

```
Lexicon (Dictionary)          Posting Lists
term -> df, cf, pointer  ->  [doc_id: tf, doc_id: tf, ...]

"computer" (df=3) -> [(doc1, 5), (doc7, 2), (doc15, 1)]
"science"  (df=2) -> [(doc1, 3), (doc7, 4)]
```

Components: - **Lexicon:** All unique terms + statistics (document frequency df, collection frequency cf) - **Document Index:** Document statistics (length, metadata) - **Inverted Index:** Mapping from terms to documents (posting lists)

Best-Match (Coordinated Search) Algorithm:

```
For each document I, Score(I) = 0
For each query term:
    Search the vocabulary (lexicon)
    Pull out the postings list
    For each document J in the list:
        Score(J) = Score(J) + weight
```

> **Exam Tip:** Draw and explain the inverted index structure. Know the difference between df (document frequency) and cf (collection frequency).

---

3. Term Weighting

Zipf's Law

Zipf's Law: The rank (r) of a word times its frequency (f) is approximately constant.

```
r * f = k    or    r * Pr = A (where A ~ 0.1 for English)
```

- **Power law:** y = kx^c (Zipf: c = -1)
- A few words occur very frequently; many words occur rarely
- On a log-log plot, power laws give a straight line

Consequences: - Very frequent words (stopwords) are poor discriminators - Very rare words (hapax legomena) don't describe content well - **Medium-frequency words are most useful** for retrieval

Heaps' Law (Vocabulary Growth)

```
V = K * n^beta
```

Where V = vocabulary size, n = number of words in corpus, K ~ 10-100, beta ~ 0.5

- Vocabulary grows sublinearly with corpus size
- New words always appear (proper names, typos, neologisms)

Term Frequency (TF)

Raw TF: Simply count occurrences of term t in document d: `tf(t,d)`

Sublinear TF (Log TF): Dampens the effect of very high frequencies

```
w(t,d) = 1 + log(tf(t,d))    if tf(t,d) > 0
w(t,d) = 0                    if tf(t,d) = 0
```

Rationale (Luhn 1957): "The frequency of a word occurrence in a document furnishes a useful measurement of word significance."

Inverse Document Frequency (IDF)

IDF measures how rare/informative a term is across the collection.

```
idf(t) = log(N / df(t))
```

Where N = total documents, df(t) = number of documents containing term t.

- Common terms (high df) get **low** IDF
- Rare terms (low df) get **high** IDF

- Captures the **discriminative power** of a term

TF-IDF Weighting

```
w(t,d) = tf(t,d) * idf(t)
       = tf(t,d) * log(N / df(t))
```

With log TF:

```
w(t,d) = (1 + log(tf(t,d))) * log(N / df(t))
```

Intuition: - High weight when term occurs often in the document (high TF) AND is rare in the collection (high IDF) - Low weight when term is common across documents OR rare in the document

> **Exam Tip:** Be able to calculate TF-IDF by hand. Know why both TF and IDF matter -- TF alone would overweight common terms, IDF alone ignores document-level importance.

---

4. Vector Space Model

Core Idea

Documents and queries are represented as **vectors in a t-dimensional space** (t = number of unique terms). Each dimension corresponds to a term weight (e.g., TF-IDF).

```
Document d = (w1, w2, ..., wt)
Query    q = (w1, w2, ..., wt)
```

Key Assumption: Documents "close together" in vector space are about the same topic.

Cosine Similarity

The standard similarity measure in the Vector Space Model:

```
cos(q, d) = (q . d) / (|q| * |d|)

            sum(wq_i * wd_i)
          = ------------------
            sqrt(sum(wq_i^2)) * sqrt(sum(wd_i^2))
```

- **Range:** 0 (orthogonal/unrelated) to 1 (identical direction)
- Normalizes for document length (unlike dot product)
- cos(0 degrees) = 1, cos(90 degrees) = 0

Worked Example

```
D1 = (0.5, 0.8, 0.3)
D2 = (0.9, 0.4, 0.2)
Q  = (1.5, 1.0, 0.0)

Dot products:
Q . D1 = 1.5*0.5 + 1.0*0.8 + 0*0.3 = 0.75 + 0.80 = 1.55
Q . D2 = 1.5*0.9 + 1.0*0.4 + 0*0.2 = 1.35 + 0.40 = 1.75

Magnitudes:
|Q|  = sqrt(2.25 + 1.0 + 0)     = sqrt(3.25)  = 1.803
|D1| = sqrt(0.25 + 0.64 + 0.09) = sqrt(0.98)  = 0.990
|D2| = sqrt(0.81 + 0.16 + 0.04) = sqrt(1.01)  = 1.005

cos(Q, D1) = 1.55 / (1.803 * 0.990) = 1.55 / 1.785 = 0.868
cos(Q, D2) = 1.75 / (1.803 * 1.005) = 1.75 / 1.812 = 0.966

Ranking: D2 > D1
```

Other Similarity Measures

Measure	Formula	Notes
Dot Product	sum(wq_i * wd_i)	No length normalization
Matching Coefficient	Count of shared non-zero terms	Ignores weights
Dice Coefficient	2|X intersect Y| / (|X| + |Y|)	Normalizes by total non-zero entries
Cosine	dot(q,d) / (|q| * |d|)	Standard choice

Query Term Weighting

- **Short queries (web):** w_kq = idf_k (no TF since terms rarely repeat)
- **Long queries (feedback):** w_kq = tf_kq * idf_k

Advantages and Disadvantages

Advantages	Disadvantages
Simple geometric interpretation	Term independence assumption
Partial matching (ranked results)	No theoretical foundation for similarity
Term weighting improves results	No guidance on optimal cutoff
Works well in practice	Missing semantic understanding

> **Exam Tip:** Be able to compute cosine similarity step by step. Understand why length normalization matters.

---

5. Evaluation Methodology

Why Evaluate?

1. **Economic:** Users/buyers need to know system effectiveness
2. **Scientific progress:** Measure and compare improvements
3. **Verification:** Confirm a system works as intended

Test Collections (Cranfield Paradigm)

A test collection consists of: 1. **Documents** (corpus) 2. **Queries** (topics) 3. **Relevance judgments** (qrels) -- which documents are relevant to which queries

TREC (Text REtrieval Conference): - Started 1992, organized by NIST (USA) - Introduced standard evaluation methodology - Uses **pooling** to make evaluation feasible

Pooling

Since exhaustively judging all documents is impractical: 1. Take top-k results from multiple systems 2. Merge into a pool, remove duplicates 3. Have assessors judge only the pooled documents 4. Assume unjudged documents are non-relevant

Core Metrics

Precision and Recall

```
Precision = |Relevant ∩ Retrieved| / |Retrieved|     (How many retrieved docs are relevant?
Recall    = |Relevant ∩ Retrieved| / |Relevant|       (How many relevant docs are retrieved
```

Trade-off: Increasing recall typically decreases precision and vice versa.

Precision at Rank k (P@k)

```
P@k = (Number of relevant documents in top k) / k
```

- Popular in web search (P@10 = precision of first results page)
- Does NOT consider the position of relevant docs within top k

Average Precision (AP)

```
AP = (1/|R|) * sum over rank k [P(k) * rel(k)]
```

Where rel(k) = 1 if document at rank k is relevant, 0 otherwise. P(k) = precision at cutoff k.

- **Single-value summary** of a ranking for one query
- Emphasizes retrieving relevant documents early
- Non-retrieved relevant documents contribute 0 to the sum

Worked Example:

```
Ranking: [R, N, R, N, R, N, N, R, N, N]  (R=relevant, N=non-relevant)
Total relevant = 6 (4 retrieved + 2 not retrieved)

P@1 = 1/1 = 1.000  (relevant -> count)
P@3 = 2/3 = 0.667  (relevant -> count)
P@5 = 3/5 = 0.600  (relevant -> count)
P@8 = 4/8 = 0.500  (relevant -> count)

AP = (1/6) * (1.0 + 0.667 + 0.600 + 0.500 + 0 + 0) = (1/6) * 2.767 = 0.461
```

Mean Average Precision (MAP)

```
MAP = (1/|Q|) * sum over all queries [AP(q)]
```

- Average of AP across all queries
- **Single number summarizing system effectiveness**
- Most widely used metric in TREC evaluations

Discounted Cumulative Gain (DCG) and NDCG

For **graded relevance** (e.g., 0, 1, 2, 3):

```
DCG@k = rel(1) + sum from i=2 to k [rel(i) / log2(i)]

      (Alternative formula commonly used:)
DCG@k = sum from i=1 to k [(2^rel(i) - 1) / log2(i + 1)]
```

Normalized DCG (NDCG):

```
NDCG@k = DCG@k / IDCG@k
```

Where **IDCG@k** = DCG of the ideal (perfect) ranking.

- Range: 0 to 1 (1 = perfect ranking)
- Handles graded relevance (not just binary)
- Discounts documents lower in the ranking

Worked Example (DCG with formula 2):

```
Ranking relevance grades: [3, 2, 3, 0, 1, 2]

DCG@6 = (2^3-1)/log2(2) + (2^2-1)/log2(3) + (2^3-1)/log2(4) + (2^0-1)/log2(5) + (2^1-1)/log
      = 7/1 + 3/1.585 + 7/2 + 0/2.322 + 1/2.585 + 3/2.807
      = 7.0 + 1.893 + 3.5 + 0 + 0.387 + 1.069
      = 13.849

Ideal ranking: [3, 3, 2, 2, 1, 0]
IDCG@6 = 7/1 + 7/1.585 + 3/2 + 3/2.322 + 1/2.585 + 0/2.807
       = 7.0 + 4.416 + 1.5 + 1.292 + 0.387 + 0
       = 14.595

NDCG@6 = 13.849 / 14.595 = 0.949
```

Reciprocal Rank (RR) and MRR

```
RR = 1 / rank of first relevant document
MRR = (1/|Q|) * sum [1/rank_i]
```

- Used for navigational queries (known-item search)

Statistical Significance

- Use **paired t-test** or **Wilcoxon signed-rank test** to compare systems
- $p < 0.05$ typically used as significance threshold
- Must test per-query, not just overall averages

> **Exam Tip:** Be able to compute P@k, AP, MAP, DCG, NDCG from a worked example. Know when to use each metric (binary vs. graded relevance).

---

6. Relevance Feedback & Query Expansion

The Query Formulation Problem

- Users often cannot express their information need perfectly
- First query is a "trial run"
- 16+% of queries are **ambiguous** (Song et al., 2009)
- Users lack knowledge of collection vocabulary and distribution

Types of Feedback

Type	Source	Example
Explicit	User marks documents as relevant/non-relevant	TREC assessors
Implicit	Inferred from user behavior	Clicks, dwell time
Pseudo (Blind)	Assume top-k results are relevant	No user input needed

Rocchio Algorithm (Vector Space Feedback)

The classic relevance feedback method in the Vector Space Model:

```
q_new = alpha * q_orig + beta * (1/|Dr|) * sum(d in Dr) d - gamma * (1/|Dnr|) * sum(d in Dr
```

Where: - **q_orig** = original query vector - **Dr** = set of relevant documents - **Dnr** = set of non-relevant documents - **alpha** = weight of original query (typically 1) - **beta** = weight of relevant documents (e.g., 0.75) - **gamma** = weight of non-relevant documents (e.g., 0.15)

Intuition: Move the query vector toward relevant documents and away from non-relevant ones.

Typical settings: alpha=1, beta=0.75, gamma=0.15 (weight relevant feedback more than non-relevant)

Pseudo-Relevance Feedback (PRF / Blind Feedback)

1. Run initial query
2. Assume top-k retrieved documents are relevant
3. Apply Rocchio (or other feedback method) to expand query
4. Re-run expanded query
5. **Advantage:** No user input required; fully automatic
6. **Disadvantage:** If initial results are poor (query drift), feedback amplifies errors

Query Expansion

Idea: Add related terms to the query to improve recall.

Sources of expansion terms: - Relevance feedback (explicit or pseudo) - Thesauri (manual or automatic) - Word embeddings (semantic similarity)

Term selection for expansion: - Choose terms that appear frequently in relevant documents - But are rare in the overall collection - Essentially looking for high TF in relevant docs + high IDF

Anomalous State of Knowledge (ASK)

Belkin et al. (1982): Users have difficulty defining their information need because that information represents a gap in their knowledge. The query is an attempt to describe what they don't know.

> **Exam Tip:** Be able to write and explain the Rocchio formula. Know the difference between explicit, implicit, and pseudo-relevance feedback.

---

7. Advanced Models -- BM25 & Language Models

Probability Ranking Principle (PRP)

Robertson (1977): If a system ranks documents in decreasing order of probability of relevance, the overall effectiveness will be the best obtainable.

```
Ranking criterion: p(R|D) -- probability document D is relevant to query
```

Binary Independence Model (BIM)

Assumptions: - Documents represented as binary term vectors (term present or absent) - Terms occur independently in documents

Retrieval Status Value (RSV):

```
RSV(D) = sum over query terms i where d_i=1 [log(p_i(1-s_i) / s_i(1-p_i))]
```

Where: - p_i = P(term i present | Relevant) - s_i = P(term i present | Non-Relevant)

Deriving IDF from BIM: When no relevance information is available: - Assume p_i = 0.5 (equal chance term appears in relevant docs) - Estimate s_i = df_i / N - This gives a weight proportional to IDF

BM25 (Best Match 25) -- Okapi BM25

The most important classical retrieval model. Extends BIM with term frequency and document length.

```
BM25(D, Q) = sum over query terms t [IDF(t) * (tf(t,D) * (k1 + 1)) / (tf(t,D) + k1 * (1 - b
```

Where: - **tf(t,D)** = term frequency of t in document D - **|D|** = document length - **avgdl** = average document length in collection - **k1** = term frequency saturation parameter (typically 1.2) - **b** = length normalization parameter (typically 0.75) - **IDF(t) = log((N - df(t) + 0.5) / (df(t) + 0.5))**

Key Properties of BM25: - **TF saturation:** As tf increases, the score contribution increases but saturates (controlled by k1) - **Length normalization:** Longer documents are penalized (controlled by b) - b=0: no length normalization - b=1: full normalization relative to average length - When k1=0, BM25 reduces to IDF-only - As k1 approaches infinity, BM25 approaches raw TF * IDF

Language Models for IR

Key Idea: Estimate the probability that a query was "generated" by a document's language model.

```
P(Q|D) = product over query terms t [P(t|D)]
```

Maximum Likelihood Estimate (MLE):

```
P_MLE(t|D) = tf(t,D) / |D|
```

Problem: Zero probability for terms not in document. Solution: **Smoothing**.

Smoothing Methods

Jelinek-Mercer (Linear Interpolation):

```
P(t|D) = lambda * P_MLE(t|D) + (1 - lambda) * P(t|C)
```

Where P(t|C) = cf(t) / |C| is the collection language model, lambda in [0,1].

Dirichlet Prior Smoothing:

```
P(t|D) = (tf(t,D) + mu * P(t|C)) / (|D| + mu)
```

Where mu is the Dirichlet prior parameter (typically 2000).

- **Dirichlet:** smoothing depends on document length (short docs get more smoothing)
- **Jelinek-Mercer:** smoothing is uniform regardless of document length

Query Likelihood vs. Divergence-Based Models

- **Query Likelihood:** Rank by P(Q|D)
- **KL-Divergence:** Rank by negative KL divergence between query model and document model

Divergence From Randomness (DFR)

Framework based on: a term that occurs more frequently in a document than expected by chance is informative.

```
weight(t,D) = -log2(Prob1) * (1 - Prob2)
```

- Prob1: probability of term frequency by a random process
- Prob2: risk of accepting the term as a good descriptor (normalization)

Field-Based Models

Documents have **fields** (title, body, URL, anchor text, etc.) with different importance.

BM25F: Extension of BM25 for multiple fields:

```
tf_combined = sum over fields f [w_f * tf(t, D_f) / (1 + b_f * (|D_f|/avg_f - 1))]
```

Then use tf_combined in standard BM25 formula.

Proximity Models

Terms appearing **close together** in a document are stronger evidence of relevance than terms far apart.

> **Exam Tip:** BM25 is critical. Know the formula, understand each parameter, and be able to explain the effect of changing k1 and b.

---

8. Learning to Rank

Motivation

- Many possible features for ranking (BM25, PageRank, field scores, etc.)
- How to combine them optimally? Use **machine learning**.

Generative vs. Discriminative Models

Type	Description	Examples
Generative	Model how data was produced, use Bayes rule	BIM, BM25, Language Models
Discriminative	Directly estimate P(relevant|features)	LTR models, neural rankers

- Generative models work well with few training examples
- Discriminative models usually better with enough training data

Ranking Cascades

```
1,000,000,000s docs -> Boolean filter -> 1000s docs -> BM25 -> 20 docs -> LTR re-ranking ->
```

Feature Types

```
Features = {
    Query-dependent: BM25 score, TF-IDF, field match scores
    Query-independent: PageRank, document length, URL depth, spam score
    Query features: query length, query type, number of terms
}
```

Three Approaches to Learning to Rank

1. Pointwise

- Treat each query-document pair independently
- Predict relevance score/label for each document
- Use regression or classification
- **Problem:** Ignores relative ordering between documents

2. Pairwise

- Learn to predict which of two documents should be ranked higher
- For each query, create pairs (d_i, d_j) where d_i should be ranked above d_j
- **RankSVM, RankNet, LambdaMART**
- **Reduces ranking to binary classification** of document pairs

3. Listwise

- Directly optimize a ranking metric (e.g., NDCG, MAP)
- Consider the entire ranked list
- **ListNet, AdaRank, LambdaRank**
- Most theoretically appealing but hardest to optimize

Common LTR Algorithms

Algorithm	Approach	Notes
RankSVM	Pairwise	SVM-based, minimize pairwise misorderings
RankNet	Pairwise	Neural network with cross-entropy loss
LambdaMART	Pairwise/Listwise	Gradient boosted trees; state-of-the-art for classical LTR
ListNet	Listwise	Uses Plackett-Luce probability model
Random Forest	Pointwise	Ensemble of decision trees
Coordinate Ascent	Listwise	Linear model, directly optimizes IR metric

LambdaMART

- Combines **LambdaRank** (gradient-based) with **MART** (Multiple Additive Regression Trees = gradient boosted trees)
- De facto standard in industry (used by Bing, Yahoo)
- Implicitly optimizes NDCG through lambda gradients

> **Exam Tip:** Know the three approaches (pointwise, pairwise, listwise), their differences, and be able to name at least one algorithm for each.

---

9. Web Search & Link Analysis

Web Query Taxonomy (Broder 2002)

Type	% of Queries	Goal	Example
Navigational	~20%	Reach a specific URL	"facebook login"
Informational	~48%	Learn about something	"climate change effects"
Transactional	~30%	Perform an action	"buy iPhone 15"

What Makes Web Search Difficult?

1. **Size:** Trillions of pages
2. **Diversity:** Many topics, languages, formats
3. **Dynamicity:** Content, queries, and user behavior change over time

Web Dynamics

- **Content:** ~34% of pages don't change; average change among rest: every 123 hours
- **Query dynamics:** Seasonal patterns, spikes (breaking news)
- **User behavior:** Changes over time, location-dependent

Web Ranking Features

```
Query-dependent: BM25, field matches (title, body, URL, anchor text)
Query-independent: PageRank, spam score, document quality
User signals: Click data, personalization, browsing history
Temporal: Document freshness, query trend
```

Link Analysis

Key Intuitions: 1. A hyperlink from page A to page B confers **authority** on B 2. The **anchor text** of a link describes the target page

PageRank

Invented by Brin and Page (1998). Models a "random surfer" following links.

```
PR(A) = (1 - d) + d * sum over pages T linking to A [PR(T) / C(T)]
```

Where: - **d** = damping factor (typically 0.85) - **C(T)** = number of outgoing links from page T - **(1 - d)** = probability of random jump (teleportation)

Interpretation: PageRank is the **stationary probability** of visiting a page during a random walk on the web graph.

Computation: Iterative power method: 1. Initialize all pages with equal PageRank (1/N) 2. Repeatedly apply the PageRank formula until convergence 3. Typically converges in ~50-100 iterations

Properties: - Query-independent (computed offline) - Used as a prior/feature in ranking - Higher PageRank = more authoritative page - Handles "dangling nodes" (pages with no outlinks) via teleportation

PageRank Worked Example (3 pages):

```
Pages: A, B, C    d = 0.85
Links: A->B, A->C, B->C, C->A

Initialize: PR(A) = PR(B) = PR(C) = 1/3

Iteration 1:
  PR(A) = 0.15 + 0.85 * [PR(C)/1]         = 0.15 + 0.85*(1/3)/1   = 0.433
  PR(B) = 0.15 + 0.85 * [PR(A)/2]         = 0.15 + 0.85*(1/3)/2   = 0.292
  PR(C) = 0.15 + 0.85 * [PR(A)/2 + PR(B)/1] = 0.15 + 0.85*((1/3)/2 + (1/3)/1) = 0.575

(Continue iterating until convergence)
```

HITS (Hyperlink-Induced Topic Search)

Kleinberg (1999). Query-dependent algorithm with two scores per page:

- **Hub score:** h(p) -- how good is this page at pointing to authorities?
- **Authority score:** a(p) -- how authoritative is this page on the topic?

Mutual reinforcement:

```
a(p) = sum over pages q that link to p [h(q)]    (good auths are pointed to by good hubs)
h(p) = sum over pages q that p links to [a(q)]   (good hubs point to good auths)
```

Algorithm: 1. Build a root set from query results 2. Expand to a base set by adding pages that link to/from root set 3. Iteratively compute hub and authority scores until convergence 4. Normalize after each iteration

PageRank vs. HITS:

Feature	PageRank	HITS
Query-dependent?	No	Yes
Computed	Offline	At query time
Scores	One score per page	Two scores (hub + authority)
Scope	Entire web graph	Topic-specific subgraph
Used by	Google	Teoma/Ask.com

Spam and Adversarial IR

- Web content creators actively try to manipulate rankings (SEO)
- **Link spam:** Creating fake links to boost PageRank

- **Content spam:** Keyword stuffing, cloaking
- Countermeasures: spam detection, TrustRank

> **Exam Tip:** Be able to compute PageRank for a small graph (3-4 nodes). Understand the random surfer model and damping factor. Compare PageRank and HITS.

---

10. Neural IR

Overview: The Shift from Lexical to Semantic

Traditional IR relies on **lexical matching** (exact term overlap). Neural IR enables **semantic matching** (meaning-based).

Generic Neural Relevance Model Structure

```
Query q -> Encode (Phi_Q) -> Query Representation
Document d -> Encode (Phi_D) -> Document Representation
-> Combine F(Phi_Q, Phi_D) -> Relevance Score
```

Types of Neural Relevance Models

1. Sparse Lexical Models (Traditional)

- **Encoder:** Term frequency vectors
- **Combiner:** Dot product
- **Examples:** TF-IDF Cosine, BM25, Query Likelihood
- **Cost:** O(|d|) encoding, O(1) combination per posting
- Can use inverted index for efficient retrieval

2. Learned Sparse Models

- Use neural networks to predict term importance weights
- **DeepCT:** Uses BERT to predict term weights (instead of raw TF)
- Still uses inverted index but with learned weights
- **SPLADE:** Learns sparse representations, can expand to terms not in document

3. Dense Retrieval (Bi-encoders)

- Encode query and document into **dense vectors** independently
- **Combiner:** Dot product or cosine similarity of dense vectors
- **Examples:** DPR (Dense Passage Retrieval), ANCE, ColBERT

```
score(q, d) = Phi_Q(q) . Phi_D(d)
```

- Documents can be **pre-encoded** offline (vectors stored in index)
- Query encoded at query time
- Uses **Approximate Nearest Neighbor (ANN)** search (e.g., FAISS)
- **Trade-off:** Fast retrieval but lower quality than cross-encoders

4. Cross-Encoders (Joint Models)

- Process query and document **together** through a single model
- **Full interaction** between all query and document tokens
- **Examples:** monoBERT, monoT5

```
score(q, d) = BERT([CLS] q [SEP] d [SEP])
```

- **Most effective** but very expensive (O((|q|+|d|)^2))
- Cannot pre-compute document representations
- Used only for **re-ranking** small candidate sets (typically top 100-1000)

5. Late Interaction Models (ColBERT)

- Encode query and document **separately** (like bi-encoder)
- But keep **token-level** representations (not compressed to single vector)
- **MaxSim:** For each query token, find maximum similarity to any document token

```
score(q, d) = sum over q_i [max over d_j [sim(q_i, d_j)]]
```

- **Trade-off:** Between bi-encoder efficiency and cross-encoder effectiveness

Transformer Language Models (BERT)

- **Self-attention mechanism:** Each token attends to all other tokens
- **Pre-trained** on large text corpora (Masked Language Modeling)
- **Fine-tuned** for IR tasks with relevance labels
- Input: [CLS] query tokens [SEP] document tokens [SEP]
- Cost: O((|q|+|d|)^2) for self-attention

Training Neural IR Models

Training data sources: - Manual relevance judgments (expensive, small scale) - Click logs (noisy but large scale) - **Hard negatives:** Documents that are similar but not relevant (improve training)

Loss functions: - **Pointwise:** Binary cross-entropy (relevant vs. non-relevant) - **Pairwise:** Hinge loss, RankNet loss - **Listwise:** ListNet, LambdaRank

Knowledge distillation: Train a smaller/faster model to mimic a larger/more accurate one.

The Efficiency-Effectiveness Trade-off

```
Effectiveness:  Cross-encoder > Late interaction > Bi-encoder > Learned sparse > BM25
Efficiency:     BM25 > Learned sparse > Bi-encoder > Late interaction > Cross-encoder
```

Typical pipeline:

```
Corpus -> BM25 (fast, retrieve 1000) -> Dense retrieval (retrieve 100) -> Cross-encoder (re
```

> **Exam Tip:** Know the four main types of neural models (sparse, dense bi-encoder, cross-encoder, late interaction), their trade-offs, and typical use in a multi-stage pipeline.

11. IR Infrastructure

The Efficiency Challenge

- Users expect sub-second response times
- Delays < 0.5 seconds impact business metrics
- At web scale: 1% increase in response time may require 1000 additional servers

Efficient Infrastructure Techniques

1. Caching

- **Result caching:** Store complete results for frequent queries
- **Posting list caching:** Cache frequently accessed posting lists
- Exploits power-law distribution of queries (few queries are very popular)

2. Static Index Pruning

Remove postings that are unlikely to affect top-k results **before** query time: - Remove postings with very low term weights - Keep only the most important postings per term - Reduces index size but may hurt recall

3. Query Evaluation Strategies

TAAT (Term-At-A-Time): - Process one term's entire posting list before moving to next term - Accumulates scores in a document-score array - **Advantage:** Sequential access to posting lists - **Disadvantage:** Large memory for accumulators (one per document)

DAAT (Document-At-A-Time): - Process all terms for one document before moving to next - Posting lists traversed in parallel (requires sorted by doc ID) - **Advantage:** Only need to maintain top-k heap (small memory) - **Disadvantage:** Requires posting lists sorted by doc ID

4. Dynamic Pruning

Skip scoring documents that cannot make it into the top-k:

MaxScore (Turtle & Flood, 1995): - Pre-compute maximum contribution of each term - Skip documents that cannot exceed current k-th score even with maximum contributions from remaining terms

WAND (Weighted AND): - More aggressive pruning than MaxScore - Uses upper bounds on term contributions to skip document blocks - **Pivot-based:** find next document that could potentially score high enough

5. Index Compression

Posting list compression: - Store **d-gaps** (differences between consecutive doc IDs) instead of raw doc IDs - Smaller numbers compress better - **Variable-byte encoding (VByte):** Use 1-4 bytes per integer - **Gamma/Delta codes:** Optimal for geometric distributions - **PForDelta:** Block-based compression, very fast decompression

Benefits: - Smaller index = less I/O - Faster decompression than disk read time - More of the index fits in memory/cache

Distributed IR

Document partitioning (horizontal): - Each server holds a subset of documents - Query sent to all servers; results merged - **Advantage:** Parallelism reduces latency - **Disadvantage:** Every query hits every server

Term partitioning (vertical): - Each server holds posting lists for a subset of terms - Query terms routed to appropriate servers - **Advantage:** Only relevant servers activated - **Disadvantage:** Load imbalance (popular terms overload servers)

In practice: Document partitioning is preferred (better load balancing).

Replication and Tiering

- **Replication:** Multiple copies of each partition for fault tolerance and throughput
- **Tiering:** Fresh/popular documents on fast tier; old/rare on slower tier

> **Exam Tip:** Know TAAT vs. DAAT, understand dynamic pruning (MaxScore/WAND), and document vs. term partitioning trade-offs.

---

12. Online Evaluation

Three Families of Evaluation

Method	Description	Pros	Cons
Offline	Test collections (docs, queries, qrels)	Reproducible, controlled	Expensive assessors, out of context
User Study	Small group completes tasks	Detailed user data, task-based	Expensive, small scale, hard to generalize
Online	Observe real users in production	Real users, real queries, high fidelity	Noisy, needs lots of data, biases

Online Evaluation Assumption

Observable user behavior reflects relevance. This gives high fidelity (real users, real needs) but introduces noise and biases.

Biases in Online Data

Bias	Description
Position bias	Users more likely to click higher-ranked results
Contextual bias	Clicks depend on surrounding results
Attention bias	Visually prominent results get more clicks
Accidental clicking	Random/unintentional clicks
Malicious clicking	Deliberate manipulation

A/B Testing

Procedure: 1. Randomly split users into two groups 2. **Control (A):** existing system 3. **Treatment (B):** changed system 4. Measure metrics of interest 5. Run statistical tests to confirm significance

Metrics used: - **Abandonment rate** (% queries with no click) - **Mean Reciprocal Rank** of clicks - **Click-through rate** - **Time to first click** - **Sessions per user** - **Revenue/purchases**

Guidelines: - Run **A/A tests** first (sanity check -- should show no difference) - Check for confounding variables - Ensure sufficient sample size for statistical power - Don't change too many variables at once

Advantages: Demonstrates causality (changes in metrics caused by the treatment) **Disadvantages:** Requires large user base; slow; users may experience degraded performance

Interleaving

A **within-user** comparison method:

Procedure: 1. For each query, interleave results from both systems (A and B) into a single list 2. Show interleaved list to user 3. Track which system contributed each clicked result 4. The system whose results get more clicks "wins"

Team-Draft Interleaving: 1. Flip a coin for who picks first 2. Systems take turns selecting their next-highest-ranked unselected document 3. Record which team contributed each document

Advantages over A/B: - Much more **sensitive** (needs fewer queries to detect differences) - Each user sees both systems (within-subject design) - Unbiased (randomized document ordering)

Disadvantages: - Only compares rankings (hard to test UI changes) - Cannot measure absolute metrics (only relative preference)

Click Models

Purpose: Model user browsing and clicking behavior to debias click data.

Cascade Model: - User examines results from top to bottom - At each position, user either clicks (and may stop) or skips - P(click) depends on relevance and position

Position-Based Model: - P(click) = P(examine position) * P(click | examined, relevant) - Separates position effect from relevance effect

> **Exam Tip:** Compare A/B testing vs. interleaving (sensitivity, what they can measure). Know the main biases in click data.

---

13. Exam Strategy & Formula Sheet

Exam Strategy

1. **Read all questions first** (2 minutes). Identify which topics each covers.
2. **Allocate time proportionally.** 90 minutes for ~4-5 questions = ~18-22 min each.
3. **Start with your strongest question** to build confidence.
4. **Show working** for any calculation -- partial credit is awarded.
5. **Define terms before using them** in explanations.
6. **Draw diagrams** where appropriate (inverted index, vector space, PR iteration).
7. **Use examples** to illustrate concepts.
8. **Comparison questions:** Use tables with clear criteria.

Common Question Types

- Define X and explain its significance in IR
- Calculate metric/score given data (P@k, AP, NDCG, BM25, cosine, PageRank)
- Compare and contrast two approaches (advantages/disadvantages table)
- Explain a process/algorithm step by step
- Discuss trade-offs (effectiveness vs. efficiency)

CLOSED BOOK FORMULA SHEET

Term Weighting

```
TF-IDF:          w(t,d) = tf(t,d) * log(N / df(t))
Log TF:          w(t,d) = (1 + log(tf(t,d))) * log(N / df(t))    [if tf > 0]
IDF:             idf(t) = log(N / df(t))
```

Vector Space Model

```
Cosine Similarity:   cos(q,d) = sum(wq_i * wd_i) / (sqrt(sum(wq_i^2)) * sqrt(sum(wd_i^2)))
Dot Product:         score = sum(wq_i * wd_i)
Dice Coefficient:    2|X intersect Y| / (|X| + |Y|)
```

BM25

```
BM25(D,Q) = sum_t [ IDF(t) * tf(t,D)*(k1+1) / (tf(t,D) + k1*(1 - b + b*|D|/avgdl)) ]

IDF(t) = log( (N - df(t) + 0.5) / (df(t) + 0.5) )

Typical: k1 = 1.2, b = 0.75
```

Language Models

```
MLE:                P(t|D) = tf(t,D) / |D|
Jelinek-Mercer:     P(t|D) = lambda * tf(t,D)/|D| + (1-lambda) * cf(t)/|C|
Dirichlet:          P(t|D) = (tf(t,D) + mu * P(t|C)) / (|D| + mu)
Query Likelihood:   P(Q|D) = product_t P(t|D)^tf(t,Q)
```

Evaluation Metrics

```
Precision:      P = |Rel intersect Ret| / |Ret|
Recall:         R = |Rel intersect Ret| / |Rel|
P@k:            P@k = (# relevant in top k) / k
F1:             F1 = 2*P*R / (P+R)

AP:             AP = (1/|Rel|) * sum_k [P(k) * rel(k)]
MAP:            MAP = (1/|Q|) * sum_q AP(q)

DCG@k:          DCG@k = sum_{i=1}^{k} (2^rel(i) - 1) / log2(i+1)
NDCG@k:         NDCG@k = DCG@k / IDCG@k

RR:             RR = 1 / rank_first_relevant
MRR:            MRR = (1/|Q|) * sum_q RR(q)
```

Rocchio (Relevance Feedback)

```
q_new = alpha*q_orig + beta*(1/|Dr|)*sum(d in Dr) - gamma*(1/|Dnr|)*sum(d in Dnr)

Typical: alpha=1, beta=0.75, gamma=0.15
```

PageRank

```
PR(A) = (1-d) + d * sum_{T->A} PR(T)/C(T)

d = damping factor (typically 0.85)
C(T) = number of outlinks from T
Initialize: PR = 1/N for all pages
```

HITS

```
Authority:  a(p) = sum_{q->p} h(q)
Hub:        h(p) = sum_{p->q} a(q)
Normalize after each iteration.
```

Zipf's Law & Heaps' Law

```
Zipf:    r * f = k    (rank * frequency = constant)
Heaps:   V = K * n^beta   (vocabulary growth, beta ~ 0.5)
```

Neural IR Cost Comparison

```
Model Type          | Encode Cost    | Combine Cost  | Pre-compute docs?
--------------------|----------------|---------------|------------------
Lexical (BM25)      | O(|d|)         | O(1)          | Yes (inverted index)
Dense Bi-encoder    | O(|d|^2)       | O(1)          | Yes (ANN index)
Late Interaction    | O(|d|^2)       | O(|q|*|d|)    | Partial
Cross-encoder       | O((|q|+|d|)^2) | N/A (joint)   | No
```

Index Compression

```
D-gaps: Store differences between consecutive doc IDs
  e.g., [1, 5, 9, 13] -> [1, 4, 4, 4]
Variable-byte (VByte): 1-4 bytes per integer, 7 bits payload per byte
```

Query Processing

```
TAAT: Process one term at a time (all docs for term 1, then term 2, ...)
DAAT: Process one doc at a time (all terms for doc 1, then doc 2, ...)
MaxScore: Skip docs that cannot reach top-k threshold
WAND: Pivot-based skipping using upper bounds
```

End of COMPSCI5011 Information Retrieval Exam Notes -- University of Glasgow, 2026